

Agent AI DevSecOps

Telegram Bot

Rapport Technique Complet

Realise par

SALMA EZZAIMI

Architecture et Implementation
Pilotage DevSecOps via Telegram

Version 1.0
Juin 2026

Table des matieres

1. Introduction

2. Architecture du Systeme

2.1 Vue d'ensemble

2.2 Composants principaux

2.3 Flux de donnees

3. Conception et Implementation

3.1 Bot Telegram

3.2 Agent AI

3.3 Gestionnaire de Pipeline

3.4 Orchestrateur de Securite

3.5 Gestionnaire de Deploiement

3.6 Systeme de Notifications

4. Securite DevSecOps

4.1 Scan SAST

4.2 Scan des Dependances

4.3 Scan Docker

4.4 Detection de Secrets

4.5 Analyse AI des Vulnerabilites

5. Interface Telegram — Demonstration

5.1 Creation du bot via BotFather

5.2 Message d'accueil et commandes

5.3 Guide d'utilisation

5.4 Statut des pipelines

5.5 Rapports de securite

6. Infrastructure et Deploiement

7. Tests et Validation

8. Conclusion et Perspectives

1. Introduction

Le contexte actuel du developpement logiciel exige une integration toujours plus etroite entre les pratiques de developpement (Dev), les operations (Ops) et la securite (Sec). L'approche DevSecOps consiste a integrer la securite a chaque etape du cycle de vie logiciel, des la conception jusqu'au deploiement en production. Cependant, la gestion des pipelines DevSecOps reste souvent un processus complexe qui necessite des interfaces techniques specialisees, ce qui limite l'accessibilite et la reactivite des equipes face aux incidents de securite.

Ce rapport presente la conception, l'architecture et l'implementation d'un Agent AI DevSecOps pilote entierement via Telegram. Ce systeme innovant permet aux equipes de developpement et de securite de controler l'integralite d'un pipeline DevSecOps directement depuis une interface de messagerie instantanee. L'agent integre des capacites d'intelligence artificielle pour le traitement du langage naturel, l'analyse des vulnerabilites et la generation de recommandations de securite, tout en offrant une interface intuitive via des commandes Telegram et des boutons interactifs.

L'objectif principal de ce projet est de democratise l'accès aux outils DevSecOps en eliminant les barrieres techniques liees a l'utilisation de lignes de commande ou d'interfaces web complexes. En utilisant Telegram comme canal d'interaction principal, l'agent permet une gestion rapide et efficace des pipelines CI/CD, des scans de securite et des deploiements, directement depuis un smartphone ou un ordinateur de bureau. Cette approche offre une reactivite accrue face aux incidents de securite et facilite la collaboration entre les membres de l'equipe, quel que soit leur niveau technique.

Fonctionnalites principales

Categorie	Fonctionnalite	Description
Pipeline CI/CD	Lancer/Arreter/Relancer	Controle complet des pipelines GitLab
Pipeline CI/CD	Statut et Logs	Consultation en temps reel des etats et traces
Securite	Scan SAST	Analyse statique du code source avec Bandit
Securite	Scan Dependances	Detection des vulnerabilites dans les packages (Safety)
Securite	Scan Docker	Analyse des images conteneurs avec Trivy

Categorie	Fonctionnalite	Description
Securite	Detection Secrets	Recherche de cles et mots de passe exposes (TruffleHog)
Deploiement	Staging/Production	Deploiement automatise des applications
Intelligence	Agent AI	Traitement du langage naturel et analyse de securite
Notifications	Alertes Telegram	Notification en temps reel des evenements critiques

2. Architecture du Systeme

2.1 Vue d'ensemble

L'architecture de l'Agent AI DevSecOps est concue selon une approche modulaire et orientee microservices. Chaque composant fonctionnel est encapsule dans un module independant, permettant une maintenance aisee, une evolution progressive et une tolerance aux pannes. Le systeme repose sur une communication asynchrone entre les differents composants, garantissant des performances optimales et une reactivite maximale aux requetes des utilisateurs.

L'architecture se decompose en six couches principales : la couche de presentation (Telegram Bot), la couche d'intelligence (AI Agent), la couche de gestion des pipelines (Pipeline Manager), la couche de securite (Security Orchestrator), la couche de deploiement (Deploy Manager) et la couche de notification (Notifier). Chaque couche communique avec les autres via des interfaces bien definies, suivant le principe de separation des responsabilites. Cette approche permet de remplacer ou de mettre a jour un composant sans affecter les autres parties du systeme.

Schema d'architecture

Le schema ci-dessous illustre le flux de communication entre les differents composants du systeme. L'utilisateur Telegram envoie des commandes qui sont traitees par le Bot, analysees par l'Agent AI, puis routees vers les gestionnaires de pipeline, de securite ou de deploiement. Les resultats sont renvoyes via Telegram et les notifications sont emises pour les evenements critiques.

```
Utilisateur Telegram --> Telegram Bot --> AI Agent (NLP)
|
+-----+-----+
| | |
| Pipeline Manager Security Orchestrator Deploy Manager
| | |
| GitLab API Bandit/Safety/Trivy Docker Registry
| | |
+-----+-----+
|
| Notifier (Telegram)
|
| Prometheus + Grafana
```

2.2 Composants principaux

Composant	Technologie	Role
Telegram Bot	python-telegram-bot 21.x	Interface utilisateur et routage des commandes
AI Agent	NLP + OpenAI API	Comprehension du langage naturel et analyse de securite
Pipeline Manager	httpx + GitLab API v4	Orchestration des operations CI/CD via GitLab
Security Orchestrator	Bandit, Safety, Trivy, TruffleHog	Coordination des scans de securite multi-outils
Deploy Manager	GitLab CI/CD + Docker	Gestion des deploiements staging/production
Notifier	Telegram API	Envoi d'alertes et notifications en temps reel
Redis	Redis 7 Alpine	Cache et gestion d'etat distribue
Prometheus	Prometheus 2.51	Collecte et stockage des metriques systeme
Grafana	Grafana 10.4	Visualisation des metriques et dashboards

2.3 Flux de donnees

Le flux de donnees au sein du systeme suit un schema clair et securise. Lorsqu'un utilisateur envoie un message via Telegram, celui-ci est d'abord recu par le Telegram Bot qui procede a l'authentification de l'utilisateur en verifiant son identifiant Telegram par rapport a une liste blanche configuree. Une fois l'utilisateur authentifie, le message est analyse par l'AI Agent qui determine l'intention de l'utilisateur en utilisant un systeme de reconnaissance de patterns base sur des expressions regulieres couvrant plus de douze categories d'intentions.

Selon l'intention detectee, la requete est routee vers le composant approprie : le Pipeline Manager pour les operations CI/CD, le Security Orchestrator pour les scans de securite, ou le Deploy Manager pour les deploiements. Chaque composant execute l'action demandee en communiquant avec les services externes (GitLab API, outils de scan, registre Docker) de maniere asynchrone. Les resultats sont ensuite formates et renvoyes a l'utilisateur via Telegram, tandis que le systeme de notification envoie des alertes complementaires pour les evenements critiques.

L'ensemble du flux est conçu pour être entièrement non-bloquant, grâce à l'utilisation d'asyncio comme runtime d'exécution. Cela signifie que le bot peut traiter plusieurs requêtes simultanément sans qu'une opération longue (comme un scan de sécurité complet) ne bloque les autres interactions. Les résultats des scans sont également mis en cache en mémoire pour permettre une analyse AI ultérieure sans avoir à relancer le scan complet.

3. Conception et Implementation

3.1 Bot Telegram

Le bot Telegram constitue la couche de presentation du systeme et le point d'entree unique pour toutes les interactions utilisateur. Il est construit sur la bibliotheque python-telegram-bot version 21.x, qui offre une API asynchrone complete pour l'interaction avec la plateforme Telegram. Le bot prend en charge trois types d'interactions : les commandes slash (/start, /status, /scan, etc.), les callbacks de claviers interactifs (Inline Keyboards) et le traitement du langage naturel pour les messages libres.

Le systeme d'authentification repose sur une liste blanche d'identifiants Telegram configures via la variable d'environnement TELEGRAM_ALLOWED_USERS. Chaque requete entrante est verifiee par un decorateur d'autorisation qui compare l'ID de l'utilisateur avec la liste autorisee. En cas d'acces non autorise, un message d'erreur est renvoye et la tentative est journalisee. Ce mecanisme assure que seul le personnel autorise peut interagir avec les pipelines et les outils de securite.

Commandes Telegram

Commande	Parametres	Description
/start	-	Message d'accueil et initialisation du bot
/status	-	Etat du dernier pipeline CI/CD
/run_pipeline	[branche]	Lancer un pipeline sur une branche donnee
/scan	[type]	Lancer un scan de securite (sast/deps/docker/secrets/all)
/deploy	[env]	Deployer en staging ou production
/logs	[pipeline_id]	Afficher les logs d'un pipeline
/help	-	Guide d'utilisation complet

3.2 Agent AI

L'Agent AI est le composant intelligent du systeme, responsable de la comprehension du langage naturel et de l'analyse des resultats de securite. Il implemente un systeme de reconnaissance d'intention base sur des expressions regulieres couvrant douze categories principales : run_pipeline, status, stop_pipeline, retry_pipeline, scan_sast, scan_deps, scan_docker, scan_secrets, scan_all, deploy_staging, deploy_production, logs et help.

Le moteur de reconnaissance d'intention utilise une hierarchie de patterns ordonnes par priorite, ou chaque pattern est une expression reguliere concue pour capturer les variations linguistiques courantes en francais et en anglais. Par exemple, l'intention 'run_pipeline' est detectee par des patterns tels que 'lance pipeline', 'demarre pipeline', 'start pipeline', 'run pipeline'. Lorsqu'un message correspond a un pattern, l'agent extrait egalement les parametres pertinents comme le nom de la branche ou l'identifiant du pipeline.

L'analyse de securite generee par l'Agent AI fournit un score de risque sur une echelle de 0 a 10, calcule en fonction du nombre et de la severite des vulnerabilites detectees. Les vulnerabilites critiques contribuent pour 4 points au score, les vulnerabilites elevees pour 3 points, les moyennes pour 2 points et les basses pour 1 point, avec un plafond a 10. L'analyse inclut egalement une description du niveau de risque, une liste des problemes critiques priorises et des recommandations de correction detaillees avec des actions immediates a entreprendre.

3.3 Gestionnaire de Pipeline

Le Pipeline Manager est le composant central pour la gestion des operations CI/CD. Il communique avec l'API GitLab REST v4 pour executer les operations de pipeline : declenchement, annulation, relance et consultation du statut. Le client GitLab est implemente de maniere asynchrone en utilisant la bibliotheque httpx, offrant des performances superieures et une gestion efficace des connexions avec un pool de connexions reutilisable.

Le gestionnaire maintient un suivi en memoire des pipelines actifs, permettant de retrouver rapidement l'etat d'un pipeline sans avoir a interroger l'API GitLab a chaque requete. Les operations supportees incluent le declenchement d'un pipeline avec des variables personnalisables, l'annulation d'un pipeline en cours d'execution, la relance d'un pipeline echoue et la recuperation des logs detailles de chaque job du pipeline, avec une limitation automatique a 500 caracteres par job pour respecter les contraintes d'affichage de Telegram.

3.4 Orchestrateur de Securite

Le Security Orchestrator coordonne l'execution des quatre types de scans de securite disponibles dans le systeme. Il implemente un modele d'execution parallele grace a asyncio.gather, permettant de lancer simultanement tous les scans lors d'une requete de scan complet. Ce modele parallele reduit considerablement le temps d'execution total d'un audit de securite complet.

Chaque scanner est encapsule dans un module independant avec une interface standardisee, facilitant l'ajout de nouveaux types de scans a l'avenir. L'orchestrateur agregge les resultats de tous les scans, calcule un score de risque global et produit une liste consolidee des vulnerabilites trieess par severite. Les resultats sont limites aux dix vulnerabilites les plus critiques pour maintenir un affichage lisible sur Telegram.

3.5 Gestionnaire de Deploiement

Le Deploy Manager gere les deploiements vers les environnements de staging et de production. Chaque deploiement declenche un pipeline CI/CD dedie avec des variables d'environnement specifiques au type de deploiement (DEPLOY_ENV, DEPLOY_TRIGGER, DEPLOY_PROD). Le gestionnaire maintient un suivi des deploiements actifs en memoire, permettant de consulter l'etat d'un deploiement a tout moment. La distinction entre les environnements est importante car le deploiement en production declenche des mecanismes de securite supplementaires.

3.6 Systeme de Notifications

Le systeme de notifications est concu pour informer les utilisateurs en temps reel des evenements critiques du pipeline DevSecOps. Il prend en charge cinq types de notifications : succes de pipeline, echec de pipeline, vulnerabilites detectees, etat du deploiement et alertes critiques. Chaque type de notification peut etre active ou desactive independamment via les variables d'environnement.

Les notifications sont formatees en Markdown pour un affichage riche dans Telegram, avec des emojis indicatifs du type d'evenement (vert pour succes, rouge pour echec, jaune pour avertissement). Le systeme inclut egalement des suggestions d'action dans les notifications d'echec, facilitant ainsi la resolution rapide des incidents.

4. Securite DevSecOps

4.1 Scan SAST

Le scan SAST (Static Application Security Testing) analyse le code source de l'application pour detecter les vulnerabilites de securite avant meme que le code ne soit execute. L'implementation utilise Bandit comme moteur d'analyse, un outil specialise pour les projets Python developpe par l'OpenStack Security Group. Bandit analyse le code source en parcourant l'arbre syntaxique abstrait (AST) de Python, ce qui lui permet de detecter des patterns de code dangereux comme les injections SQL, les deserialisations non securisees, les appels a subprocess avec shell=True, et les mots de passe codes en dur.

Le scanner SAST est configure pour ne signaler que les vulnerabilites de severite moyenne et elevee par default, afin de reduire le bruit et de concentrer l'attention sur les problemes les plus critiques. Les resultats sont structures avec des informations detaillees sur chaque vulnerabilite : identifiant Bandit, nom du test, severite, confiance, description, fichier source, numero de ligne, extrait de code et identifiant CWE. Un mode simulation est egalement disponible pour les demonstrations et les tests, generant des resultats realistes sans necessiter l'installation de Bandit.

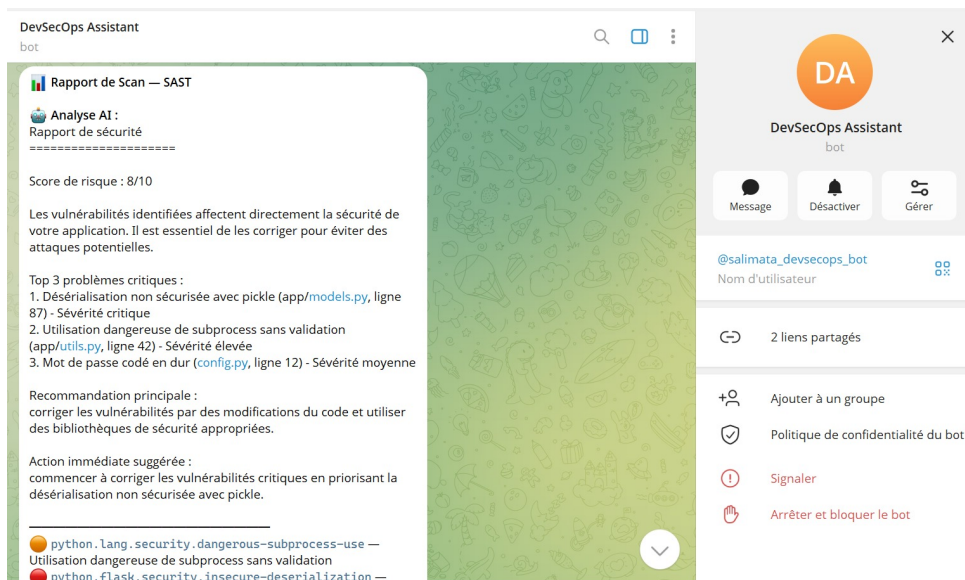


Figure 1 : Rapport de scan SAST avec analyse AI et score de risque

4.2 Scan des Dependances

Le scan des dependances verifie les bibliotheques et packages tiers utilises par l'application contre les bases de donnees de vulnerabilites connues (CVE). L'implementation utilise Safety, un outil specialise pour l'ecosysteme Python qui maintient une base de donnees actualisee des vulnerabilites affectant les packages PyPI. Le scanner detecte automatiquement le fichier

de dependances du projet (requirements.txt, Pipfile.lock ou poetry.lock) et l'analyse pour identifier les versions vulnérables.

Les résultats du scan des dépendances incluent pour chaque vulnérabilité : le nom du package affecté, la version installée, l'identifiant CVE, le niveau de sévérité, une description de la vulnérabilité et un lien vers les informations détaillées. Cette approche permet aux développeurs de prioriser les mises à jour de sécurité en fonction du niveau de risque réel.

4.3 Scan Docker

Le scan Docker analyse les images de conteneurs pour détecter les vulnérabilités dans les packages système et les bibliothèques incluses dans l'image. L'implémentation utilise Trivy comme scanner principal, avec Docker Scout comme alternative. Trivy est un scanner de vulnérabilités complet qui supporte les images OCI, les systèmes de fichiers, les dépôts Git et les fichiers de configuration Kubernetes. Il détecte les vulnérabilités dans les packages système et les packages d'application.

Le scanner Docker implémente un mécanisme de fallback automatique : si Trivy n'est pas disponible, il tente d'utiliser Docker Scout, et si aucun des deux outils n'est installé, il bascule en mode simulation qui génère des résultats réalistes pour les démonstrations. Les résultats sont structurés avec des informations sur chaque CVE trouvée : identifiant CVE, package affecté, version installée, version corrigée disponible, sévérité et titre de la vulnérabilité.

4.4 Detection de Secrets

La détection de secrets est un composant critique du système DevSecOps, chargé d'identifier les informations sensibles exposées dans le code source : clés API, tokens d'authentification, mots de passe, clés privées et autres données confidentielles. L'implémentation utilise TruffleHog comme scanner principal, avec un système de détection par expressions régulières en fallback.

Le système de détection par regex intègre des patterns pour les secrets les plus courants : clés d'accès AWS (AKIA...), clés secrètes AWS, tokens GitHub (ghp_/ghu_/ghs_), tokens GitLab (glpat-), tokens Slack (xoxb-/xoxp-), clés API Google (Alza...), clés privées (BEGIN PRIVATE KEY), tokens JWT (eyJ...) et des patterns génériques pour les mots de passe et clés API codés en dur. Le scanner parcourt les fichiers du projet en excluant les répertoires sensibles (.git, node_modules, __pycache__, venv) et analyse chaque fichier avec les extensions pertinentes.

4.5 Analyse AI des Vulnérabilités

L'analyse AI des vulnérabilités constitue une couche d'intelligence supplémentaire au-dessus des résultats bruts des scanners. Pour chaque scan, l'Agent AI génère un rapport d'analyse qui inclut un score de risque sur 10, une description qualitative du niveau de risque, la liste des problèmes critiques prioritaires et des recommandations de correction avec des actions immédiates. L'analyse s'adapte au niveau de risque détecté : un score élevé (8+) déclenche un message d'alerte recommandant une correction immédiate, tandis qu'un score faible

(0-2) felicite l'equipe pour le bon niveau de securite.

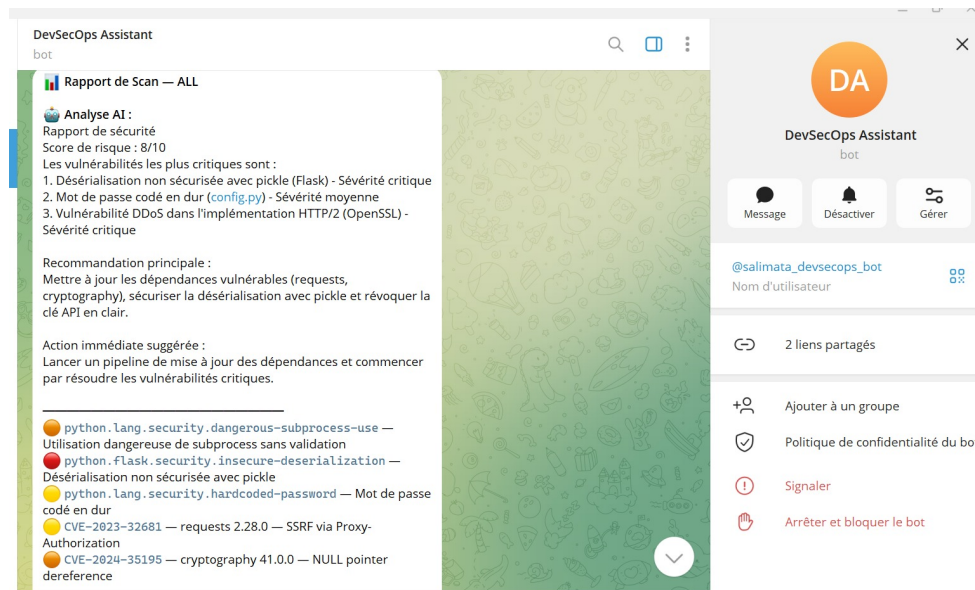


Figure 2 : Rapport de scan complet (ALL) avec vulnerabilites et recommandations

Comparaison des outils de securite

Outil	Type de scan	Langage	Severites
Bandit	SAST	Python	Low/Medium/High
Safety	Dependances	Python (PyPI)	CVE basee
Trivy	Docker/IaC	Multi-langage	Critical/High/Medium/Low
TruffleHog	Secrets	Multi-langage	Verified/Unverified

5. Interface Telegram — Demonstration

Cette section présente les captures d'écran réelles de l'interface Telegram du bot DevSecOps Assistant. Ces images illustrent le fonctionnement complet du système, depuis la création du bot via BotFather jusqu'aux rapports de sécurité détaillés avec analyse AI. Chaque capture est accompagnée d'une description détaillée des fonctionnalités démontrées.

5.1 Creation du bot via BotFather

La première étape de la mise en place du système consiste à créer le bot Telegram via le bot officiel BotFather (@BotFather). L'utilisateur commence par envoyer la commande `/newbot`, puis choisit un nom pour le bot (DevSecOps Assistant) et un nom d'utilisateur se terminant par 'bot' (salimata_devsecops_bot). BotFather fournit ensuite le token d'API nécessaire pour configurer le bot dans l'application.

Le token d'API est une cle unique qui authentifie le bot auprès de l'API Telegram. Il doit être conservé de manière sécurisée et configuré dans la variable d'environnement `TELEGRAM_BOT_TOKEN` du fichier `.env`. La capture d'écran ci-dessous montre le processus complet de création du bot, y compris le token généré par BotFather et le lien d'accès direct au bot.

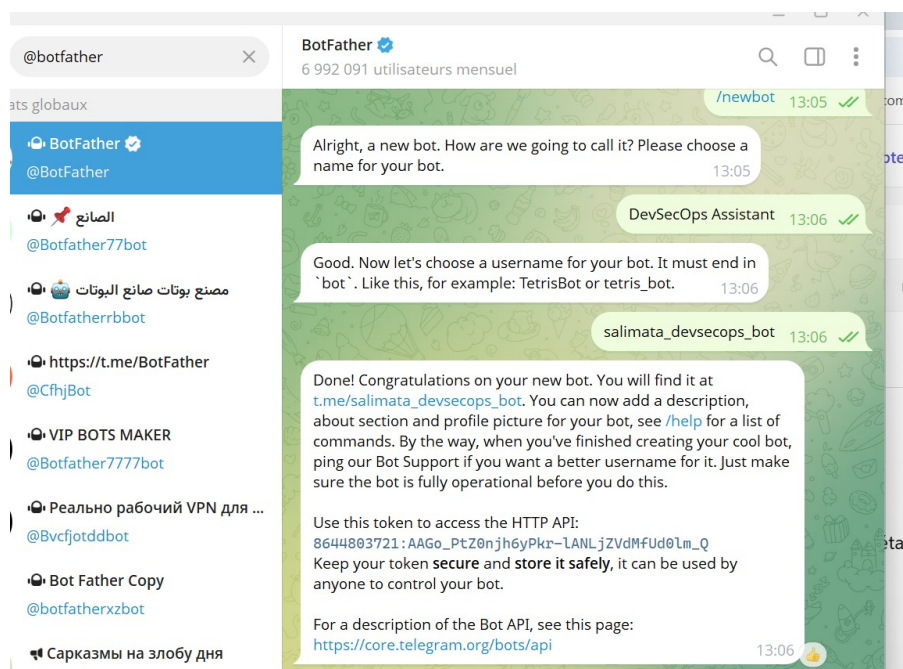


Figure 3 : Creation du bot via BotFather — nom, username et token d'API

5.2 Message d'accueil et commandes

Lorsqu'un utilisateur démarre le bot pour la première fois avec la commande /start, le bot affiche un message d'accueil personnalisé qui présente ses capacités et liste les commandes disponibles. Le message inclut également des boutons de réponse rapide (Reply Keyboard) qui permettent d'accéder aux fonctions principales sans taper de commandes : Lancer Pipeline, Statut, Scan Sécurité, Deploy, Logs et Aide. Le bot indique également qu'il comprend le langage naturel en plus des commandes slash.

Cette interface a été conçue pour être intuitive et accessible, même pour les utilisateurs qui ne sont pas familiers avec les outils DevSecOps. Les boutons de réponse rapide offrent un accès direct aux fonctionnalités les plus utilisées, tandis que les commandes slash offrent un contrôle plus fin avec des paramètres optionnels (branche, type de scan, environnement de déploiement).

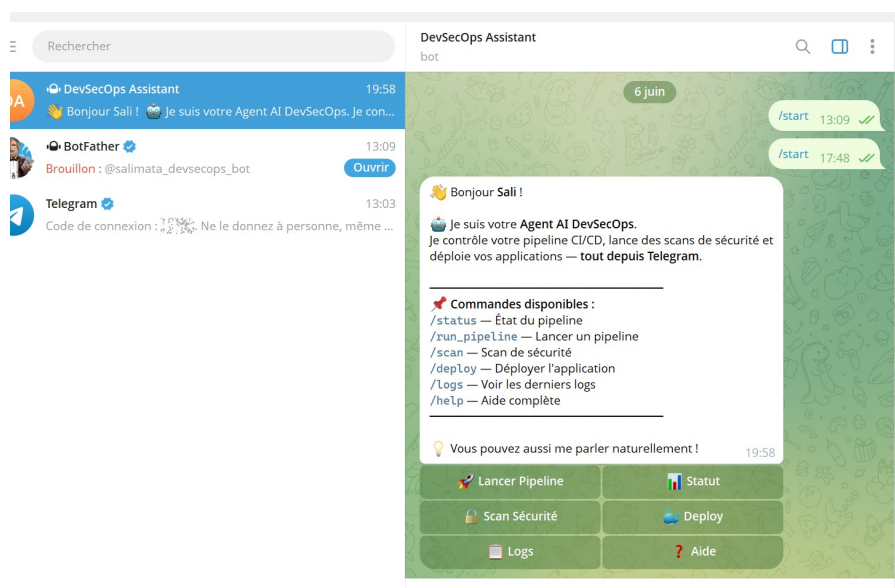


Figure 4 : Message d'accueil du bot avec commandes et boutons de réponse rapide

5.3 Guide d'utilisation

La commande /help affiche un guide d'utilisation complet organisé en trois catégories : commandes Pipeline, commandes Sécurité et commandes Déploiement. Chaque commande est accompagnée d'une description concise et des paramètres optionnels. Le guide inclut également une astuce pour l'utilisation du langage naturel, avec un exemple concret : 'Lance un scan de sécurité et déploie si tout est OK'. Cette approche pédagogique facilite la prise en main du système par les nouveaux utilisateurs.

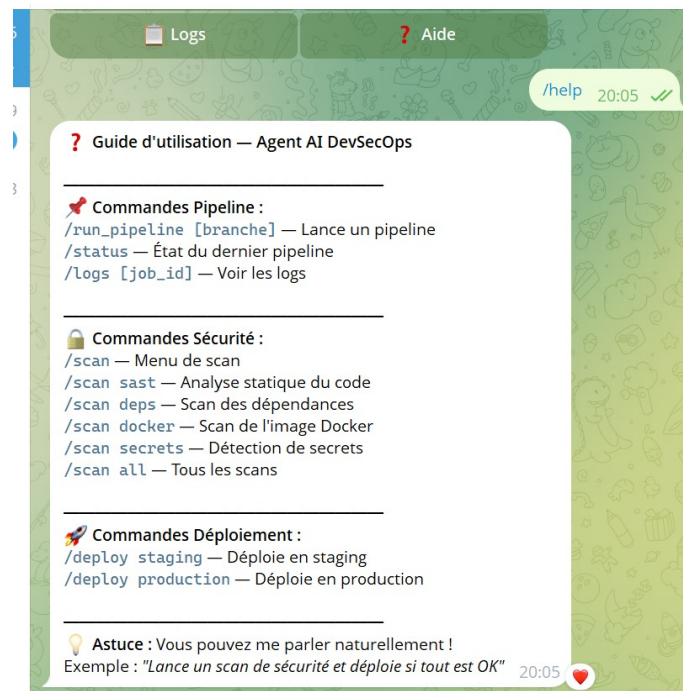


Figure 5 : Guide d'utilisation complet affiche par la commande /help

5.4 Statut des pipelines

La commande /status permet de consulter l'état du dernier pipeline CI/CD. Le bot affiche une carte d'information contenant les détails du pipeline : identifiant, statut (success/failed/running/pending), branche, déclencheur, durée et date de création. Les différents statuts sont visuellement différenciés par des emojis : coche verte pour succès, croix rouge pour échec, flèche circulaire pour en cours et sablier pour en attente.

Les captures d'écran ci-dessous montrent trois états différents du pipeline : l'état initial lorsqu'aucun pipeline n'a encore été exécuté (statut 'none'), un pipeline en échec sur la branche main, et un pipeline réussi. Chaque carte est accompagnée de boutons d'action contextuels (Relancer, Annuler, Logs, GitLab) qui permettent d'agir directement sur le pipeline sans saisir de commande supplémentaire.



Figure 6 : Statut du pipeline — Aucun pipeline exécuté (état initial)

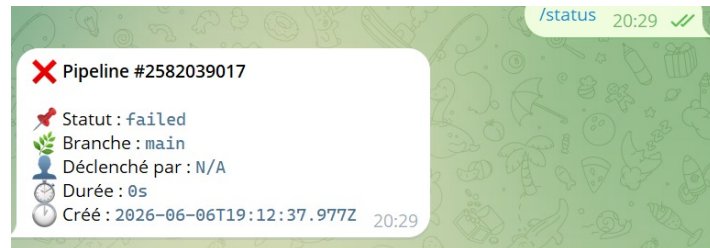


Figure 7 : Statut du pipeline — Pipeline echoue sur la branche main



Figure 8 : Statut du pipeline — Pipeline reussi sur la branche main

5.5 Rapports de securite

Les rapports de securite constituent l'une des fonctionnalites les plus importantes du bot. Apres chaque scan, le bot affiche un rapport detaille comprenant : le type de scan effectue, le score de risque, la liste des vulnerabilites detectees trieess par severite, et des boutons d'action pour re-scanner ou lancer une analyse AI plus approfondie. Les vulnerabilites sont presentees avec des emojis de couleur indiquant leur severite : rouge pour critique, orange pour elevee, jaune pour moyenne et vert pour basse.

Le rapport de scan SAST (Figure 9) presente les vulnerabilites detectees dans le code source, incluant la deserialisation non securisee avec pickle, l'utilisation dangereuse de subprocess et les mots de passe codes en dur. L'analyse AI accompagne chaque rapport d'un score de risque et de recommandations de correction priorisees. Le rapport de scan complet (Figure 10) agregge les resultats de tous les types de scans, incluant les vulnerabilites SAST, les CVE des dependances et les secrets detectes.

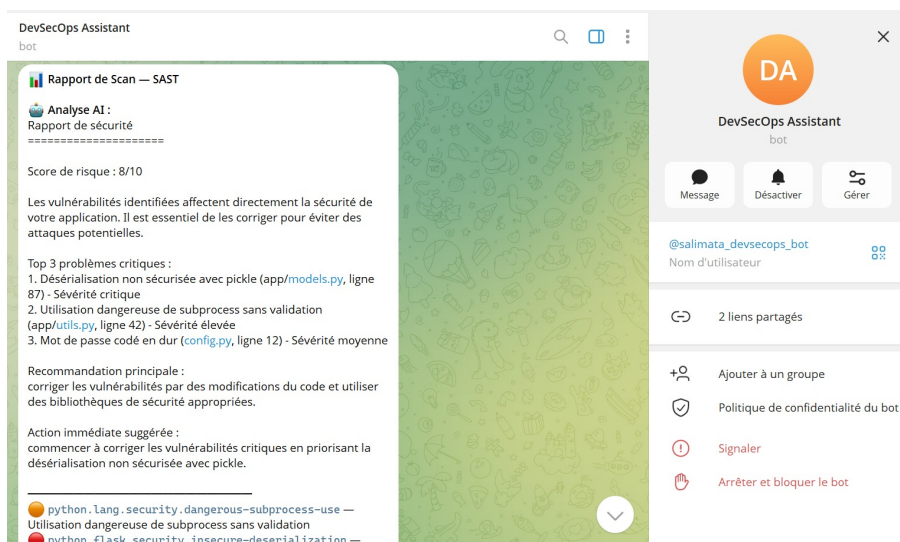


Figure 9 : Rapport de scan SAST — Analyse AI avec score de risque 8/10

The screenshot displays the DevSecOps Assistant chat window. The main content area shows a report titled "Rapport de Scan — ALL". The report includes an "Analyse AI" section with a security score of 8/10 and a list of critical vulnerabilities. The vulnerabilities listed are:

- python.lang.security.dangerous-subprocess-use — Utilisation dangereuse de subprocess sans validation
- python.flask.security.insecure-deserialization — Désérialisation non sécurisée avec pickle
- python.lang.security.hardcoded-password — Mot de passe codé en dur
- CVE-2023-32681 — requests 2.28.0 — SSRF via Proxy- Authorization
- CVE-2024-35195 — cryptography 41.0.0 — NULL pointer dereference

The report also includes a main recommendation to update vulnerable dependencies and a suggested immediate action to run a pipeline. On the right side of the chat window, there is a user profile for "DevSecOps Assistant" with a "DA" avatar, and a list of actions: "Message", "Désactiver", and "Gérer". Below the profile, there is a link to the user's profile (@salimata_devsecops_bot) and a section for "2 liens partagés". At the bottom right, there are options to "Ajouter à un groupe", "Politique de confidentialité du bo", "Signaler", and "Arrêter et bloquer le bot".

Figure 10 : Rapport de scan complet (ALL) — Vulnerabilites et recommandations

6. Infrastructure et Deploiement

6.1 Architecture Docker

L'infrastructure du systeme est entierement conteneurisee grace a Docker et orchestree via Docker Compose. Le fichier docker-compose.yml definit cinq services principaux : l'agent DevSecOps (application principale), Redis (cache et gestion d'etat), Prometheus (collecte de metriques), Grafana (visualisation) et un conteneur de mise a jour de la base de donnees Trivy. Chaque service est configure avec des politiques de redemarrage automatique (unless-stopped) et des health checks pour garantir la resilience du systeme.

Le Dockerfile utilise une construction multi-stage basee sur Python 3.12-slim, incluant l'installation des outils de securite (Bandit, Trivy) et la configuration d'un utilisateur non-root pour des raisons de securite. Les volumes Docker sont utilises pour persister les donnees de Redis, Prometheus et Grafana, ainsi que pour partager le socket Docker entre l'hote et le conteneur de l'agent (en lecture seule) pour permettre les operations de scan Docker.

Services Docker

Service	Image	Port	Role
devsecops-agent	Custom (Python 3.12)	8080	Application principale
redis	redis:7-alpine	6379	Cache et etat distribue
prometheus	prom/prometheus:v2.51	9090	Collecte de metriques
grafana	grafana/grafana:10.4	3000	Dashboards de monitoring

6.2 Configuration

La configuration du systeme est entierement basee sur des variables d'environnement, chargees a partir d'un fichier .env. Cette approche suit les principes du twelve-factor app et facilite le deploiement dans differents environnements (developpement, staging, production) sans modification du code. Les variables de configuration sont validees et typees via Pydantic Settings, qui offre une verification automatique des types, des valeurs par default et une documentation integree.

Les variables d'environnement sont organisees en six categories : Telegram (token du bot, utilisateurs autorises, chat ID admin), GitLab (URL, token d'accès, ID du projet, branche par default), Docker (URL du registre, nom de l'image, identifiants), AI Agent (cle API, modele, parametres de temperature et de tokens), Securite (fichiers de configuration Bandit, cle API Safety) et Notifications (activation/desactivation par type).

6.3 Monitoring

Le stack de monitoring integre dans le systeme repose sur Prometheus pour la collecte des metriques et Grafana pour la visualisation. Prometheus est configure pour scraper les metriques de l'agent DevSecOps (endpoint /metrics sur le port 8080) et de Redis (port 6379) a un intervalle de 15 secondes. Grafana est preconfigure avec un mot de passe admin par default et peut etre accede via le port 3000 pour creer des dashboards personnalisés.

Le monitoring couvre les aspects suivants : l'etat de sante de l'agent (verification de l'accès a l'API Telegram et GitLab), le nombre de pipelines lances, reussis et echoués, le nombre de vulnerabilites detectees par type de scan, les temps de reponse des scans de securite et des operations GitLab, et l'utilisation des ressources systeme (CPU, memoire, disque). Ces metriques permettent aux equipes d'operations de surveiller l'etat du systeme en temps reel.

7. Tests et Validation

Le systeme inclut une suite de tests unitaires couvrant les composants critiques de l'application. Les tests sont implements avec le framework pytest et la bibliotheque pytest-asyncio pour la gestion des tests asynchrones. La couverture de test porte sur quatre composants principaux : l'AI Agent, le Security Orchestrator, le Pipeline Manager et le Deploy Manager.

Les tests de l'AI Agent verifient la capacite du systeme a detecter correctement les intentions de l'utilisateur a partir de differentes formulations linguistiques. Douze cas de test couvrent les intentions principales : run_pipeline, status, scan_sast, scan_all, deploy_staging, deploy_production, logs, help et unknown. Des tests supplementaires verifient l'extraction des parametres (comme le nom de la branche) et la qualite de l'analyse de securite generee par l'IA pour differents niveaux de risque.

Les tests du Security Orchestrator verifient la disponibilite des types de scans et la gestion des types de scan invalides. Les tests du Deploy Manager valident la liste des environnements disponibles, le rejet des environnements invalides et la gestion de l'absence de deploiements. L'ensemble des tests peut etre execute avec la commande 'pytest tests/ -v' et la couverture de code peut etre generee avec 'pytest tests/ -v --cov=src --cov-report=html'.

Couverture des tests

Composant	Cas de test	Couverture
AI Agent - Intent Parser	12 tests	Reconnaissance d'intention + extraction de parametres
AI Agent - Analyse Securite	2 tests	Generation de rapports pour risque eleve et faible
Security Orchestrator	2 tests	Types de scan valides et invalides
Pipeline Manager	1 test	Etat initial des pipelines actifs
Deploy Manager	2 tests	Environnements valides et deploiements absents

8. Conclusion et Perspectives

Ce rapport a presente la conception, l'architecture et l'implementation d'un Agent AI DevSecOps pilote entierement via Telegram. Le systeme realise l'objectif de democratise l'accès aux outils DevSecOps en offrant une interface intuitive accessible depuis n'importe quel appareil equipe de Telegram. L'integration de l'intelligence artificielle pour le traitement du langage naturel et l'analyse des vulnerabilites apporte une valeur ajoutée significative par rapport aux outils traditionnels de gestion de pipelines.

L'architecture modulaire du systeme, basee sur des composants independants communiquant via des interfaces bien definies, assure la maintenabilite et l'evolubilite de la solution. Les six couches fonctionnelles (presentation, intelligence, pipeline, securite, deploiement et notification) peuvent etre modifiees ou remplacees independamment, permettant d'adapter le systeme aux besoins specifics de chaque organisation.

L'approche DevSecOps integree, combinant l'analyse statique du code (SAST), le scan des dependances, l'analyse des images Docker et la detection de secrets, offre une couverture de securite complete tout au long du cycle de vie logiciel. L'execution parallele des scans de securite optimise les temps de reponse, tandis que l'analyse AI fournit des recommandations actionnables pour prioriser les corrections.

Perspectives d'evolution

Plusieurs axes d'amelioration sont envisages pour les versions futures du systeme. Premierement, l'integration d'un modele de langage plus avance (comme GPT-4 ou Claude) pour l'Agent AI permettrait d'ameliorer significativement la comprehension du langage naturel et la qualite des analyses de securite, en passant d'un systeme base sur des expressions regulieres a un systeme base sur la comprehension semantique des requetes utilisateur.

Deuxiemement, l'ajout de support pour d'autres plateformes CI/CD (GitHub Actions, Jenkins, Azure DevOps) elargirait le champ d'application du systeme au-dela de l'ecosysteme GitLab. Troisiemement, l'implementation d'un systeme d'approbation multi-utilisateurs pour les deploiements en production renforcerait la securite du processus de deploiement en exigeant l'approbation de plusieurs membres de l'equipe avant chaque mise en production.

Quatriemement, l'integration de l'analyse dynamique (DAST) et des tests de penetration automatises completerait la couverture de securite en ajoutant des tests sur l'application en cours d'execution. Cinquiemement, l'ajout d'un systeme de chat multi-utilisateurs avec des canaux dedies par projet permettrait de gerer plusieurs projets DevSecOps en parallele, chacun avec ses propres pipelines et configurations de securite. Enfin, l'implementation d'un systeme de rollback automatique en cas de detection de vulnerabilites critiques post-deploiement renforcerait la resilience du systeme en production.